

## 4.3 Defining types for ourselves: enumerated types

Often the first thing we need to do in solving a problem is to find the right types to model the problem domain. While Haskell comes with a comprehensive set of basic data types, which we discussed in Chapter 3, and ways of building complex types from simpler ones, which we'll discuss in the coming chapters, it also makes sense to define types which directly model the problem domain. This is done with Haskell data types: this section introduces the simplest cases of these types in the context of a gaming example.

### Rock - Paper - Scissors

Two players choose one of Rock, Paper and Scissors after counting to three and making one of these gestures:

- **A clenched fist** which represents a rock.
- **A flat hand** representing a piece of paper.
- **Index and middle figure extended** which represents a pair of scissors.



If they choose the same gesture, neither wins; if not, the result is decided this way:

- Rock defeats scissors, because a rock will blunt a pair of scissors.
- Paper defeats rock, because a paper can wrap up a rock.
- Scissors defeat paper, because scissors cut paper.

Suppose that we want to model this in Haskell. One option would be to use integers, characters or strings (which we'll find out about later), but none of these is ideal, for a number of reasons.

- A choice may well be arbitrary: what numbers should we associate with 'rock', which with 'scissors'?
- The type we would choose would have lots of other elements which don't correspond to anything in the game.

So, instead, we will define a data type with three members

```
data Move = Rock | Paper | Scissors
```

where we list the members separated by a vertical bar. We can also put the different members on different lines, like this:

```
data Move = Rock |  
           Paper |  
           Scissors
```

Whatever the case, we add another line – some ‘Haskell magic’ which we’ll explain in Chapter 13 – which allows us to compare elements of this type for equality, and also to be able to see them printed out (or ‘shown’), like so:

```
data Move = Rock | Paper | Scissors
          deriving (Show,Eq)
```

Now we can begin to define functions using the `Move` type. Let’s write a function which tells us the move to beat a particular move:

```
beat :: Move -> Move

beat Rock    = Paper
beat Paper   = Scissors
beat Scissors = Rock
```

and also the move that will lose against a particular move.

```
lose :: Move -> Move

lose Rock  = Scissors
lose Paper = Rock
lose _     = Paper
```

In the definition of `lose` we’ve used a wildcard ‘`_`’ instead of `Scissors` in the final clause of the definition. That is because this clause is only matched when the others don’t, and that will only happen for the value `Scissors`.

We will come back to this example in Section 8.1 once we have covered lists in Haskell, when we’ll look at some of the strategies for playing (and winning!) Rock - Paper - Scissors.

### Exercises

**4.11** Define a data type `Result` which represents the outcome of a round of rock - paper - scissors, which will either be a win, lose or draw.

**4.12** Define a function

```
outcome :: Move -> Move -> Result
```

so that this gives the outcome of a round for the first player. For example, we should expect that `outcome Rock Scissors` should be a win.

**4.13** We have added some ‘magic’ to the `Chapter4` module to allow `QuickCheck` properties to be tested over the `Move` type. Define a `QuickCheck` property which connects the results of `beat` and `lose`.

**4.14** How would you define a `QuickCheck` property to test the `outcome` function?

## Standard types

We can see some standard types as being defined in this way. In particular, we could define `Bool` like this:

```
data Bool = False | True
          deriving (Show, Eq, Ord)
```

Deriving `Ord` means that the elements have an ordering defined on them; in this case `False < True`.

### Exercises

- 4.15** Define a type of seasons, `Season`, and give a function from seasons to temperature given by the type

```
data Temp = Cold | Hot
          deriving (Eq, Show, Ord)
```

In defining this function assume that you're in the UK.

- 4.16** Define a type `Month` and a function from this type to `Season`, assuming that you're in the northern hemisphere.

## 4.4 Recursion

Recursion is an important programming mechanism, in which a definition of a function or other object refers to the object itself. This section concentrates on explaining the idea of recursion, and why it makes sense. In particular we give two complementary explanations of how primitive recursion works in defining the factorial function over the natural numbers. In the section after this we look at how recursion is *used* in practice.

### Getting started: a story about factorials

Suppose that someone tells us that the factorial of a natural number is the product of all natural numbers from one up to (and including) that number, so that, for instance

```
fac 6 = 1*2*3*4*5*6
```

Suppose we are also asked to write down a table of factorials, where we take the factorial of zero to be one. We begin thus

| n | fac n        |
|---|--------------|
| 0 | 1            |
| 1 | 1            |
| 2 | 1*2 = 2      |
| 3 | 1*2*3 = 6    |
| 4 | 1*2*3*4 = 24 |